Rīgas Tehniskā Universitāte

Datorzinātņu un Informācijas Tehnoloģiju fakultāte

Priekšmets: Projektēšanas laboratorija

Universitātes telpu gaisa kvalitātes pārraudzības sistēma

Komanda:

Katrīna Anete Goba

Maija Rudzīte

Agnese Krūmiņa

Konrāds Rudzītis

Raivis Amoliņš

Oskars Grauzis

Rīga, 2023/2024 mācību gads

Saturs

1 Ievads 3
2 Līdzīgo risinājumu pārskats 5
3 Tehniskais risinājums 6
3.1 Prasības 6
3.2 Algoritms 7
3.3 Konceptu modelis 11
3.4 Tehnoloģiju steks 12
3.5 Programmatūras apraksts 12
4 Novērtējums 13
5 Secinājumi 15

1 Ievads

Mūsu komanda bija izvēlējusies veidot projektu Universitātes gaisa kvalitātes pārvaldības sistēma. Pēc sarunas ar "pasūtītāju" iezīmējām galvenos lietotāja stāstus. Tākā viens no stāstiem bija saslēgšana, tad realizējām 3 dažādu interfeisu sensorus, un vienu no tiem kalibrējām paši.
Lai gan mūsu komandā bija pamanāmi cilvēki ar ideālistiskiem skatījumiem, tomēr jāņem vērā, ka nav dzirdēts par plašai publikai brīvi pieejamiem nemirstības risinājumiem, kas nozīmē, ka projekta izstrādei ir tikai neliels laika resurss, kas nozīmē, ka jākoncentrējas uz MVP sasniegšanu, nevis jāiedziļinās dažādos interesantās blakusnodarbēs kā absolūtās drošības meklējumi, neskaitāmu interfeisu izmēģināšana un tamlīdzīgi.

Rezultātā ieguvām mūsuprāt ievērojami dzīvotspējīgu, pietiekami ērti lietojamu un visnotaļ viegli mērogojamu sistēmas iestrādni. Kurai nav papildus atkarība no kādas attālinātas korporācijas lēmumu mainības vai nemainības. Tātad pielietojama arī attālākos civilizācijas punktos vai sasniedzot briedumu arī citur kosmosā.

Nosakot vadlīnijas, nolēmām, ka sekošanu hype un dažādiem modes zīmoliem šeit neprioritizēsim, bet tieši pretēji - vērsīsim skatus uz pietiekami ilgi esošām sistēmām ar maksimāli iespējamo koda caurredzamību, tajā pašā laikā kā jau ļoti limitētu resursu sistēmās, neaizmirstot par resursu limitiem.

Tāpēc izvēlējāmies python, un tikai divos gadījumos: webkameras blakus mini projektā un industriālā 4-20mA standartsensora gadījumā, lietojām c++. Pirmajā gadījumā jo kamerai bija kritiski nepieciešama operatīvā atmiņa, bet sensora gadījumā - jo kontrolierim ar augstākas izšķirtspējas ADC micropython netiek uzturēts, kas nozīmē ka liekot savu portu, ļoti ievērojami daudz laika aizietu mappinga, testēšanas un citos uzdevumos, kas acīmredzami nebūtu laika resursa daudzuma ziņā samērīgi ar projekta kopējo laika resursu.

Mikrokontrolieru kods ir asinhronu uzdevumu struktūrā veidots, un pat zemākā gala kontrolieris (esp32) to spēja izpildīt, pat esot uz operatīvās atmiņas pārpildes robežas. (jāpiezīmē, ka ja kodā atstāja 5 sensorus, jau pie izpildes bija kļūda ka nepietiek RAM. Saīsinot kodu uz 2 sensoriem viss atkal strādāja).
Arī API endpointu kods ir tajā pašā asyncio satvarā veidots. Arhivators turpretī ir sinhrons, jo tas ir vienlaikus viens uzdevums, tāpēc asinhromums ir lieks. Savukārt lietotāja saskarne ir salikts projekts, sastāvošs no Grafana, kas rakstīta go valodā un pielieto react un angular javascript satvarus un no Django satvarā veidotas CRUD un maršrutēšanas lietotnes, kas pielieto DataTables javascript satvaru sarakstu kārtošanai un meklēšanai, jo Django diemžēl modeļus atlasīja ievērojami lēni.

Darba mērķi bija vairāki,

- Izveidot mājaslapas saslēgumu ar sensoru datiem
- Parādīt datus reāllaika, vēlams vizuāli viegli uztveramā formā un iezīmējot kritiskās gāzu vērtības
- Dot iespēju regulēt datu ievākšanas biežumu, arhivēšanas biežumu un dzēšanas biežumu
- Dot iespēju veidot sensoru aprakstošo datu (metadatu) datubāzi un tai piesaistītās CRUD operācijas

Novērtēšanas mērķi ir pārliecināties, ka ir funkcionāls MVP pēc iespējas vairākos lietotāja stāstos nosauko prasību aspektos, kā arī vai ir vai arī ja nav, tad vai ir samērā vienkārši un ātri realizējamas drošības prasības.

2 Līdzīgo risinājumu pārskats

- CISCO sensors Meraki MT14 (https://documentation.meraki.com/MT/MT_Datasheets/MT14_Datasheet_-_Indoor_Air_Quality_Monitor)
- Tech stack alternatīvas:
    - openHAB un kontrolieri kā mqqt klienti
    - freeRTOS uz konroliera (https://github.com/aws-iot-builder-tools/freertos-pi-pico)

3 Tehniskais risinājums

3.1 Prasības

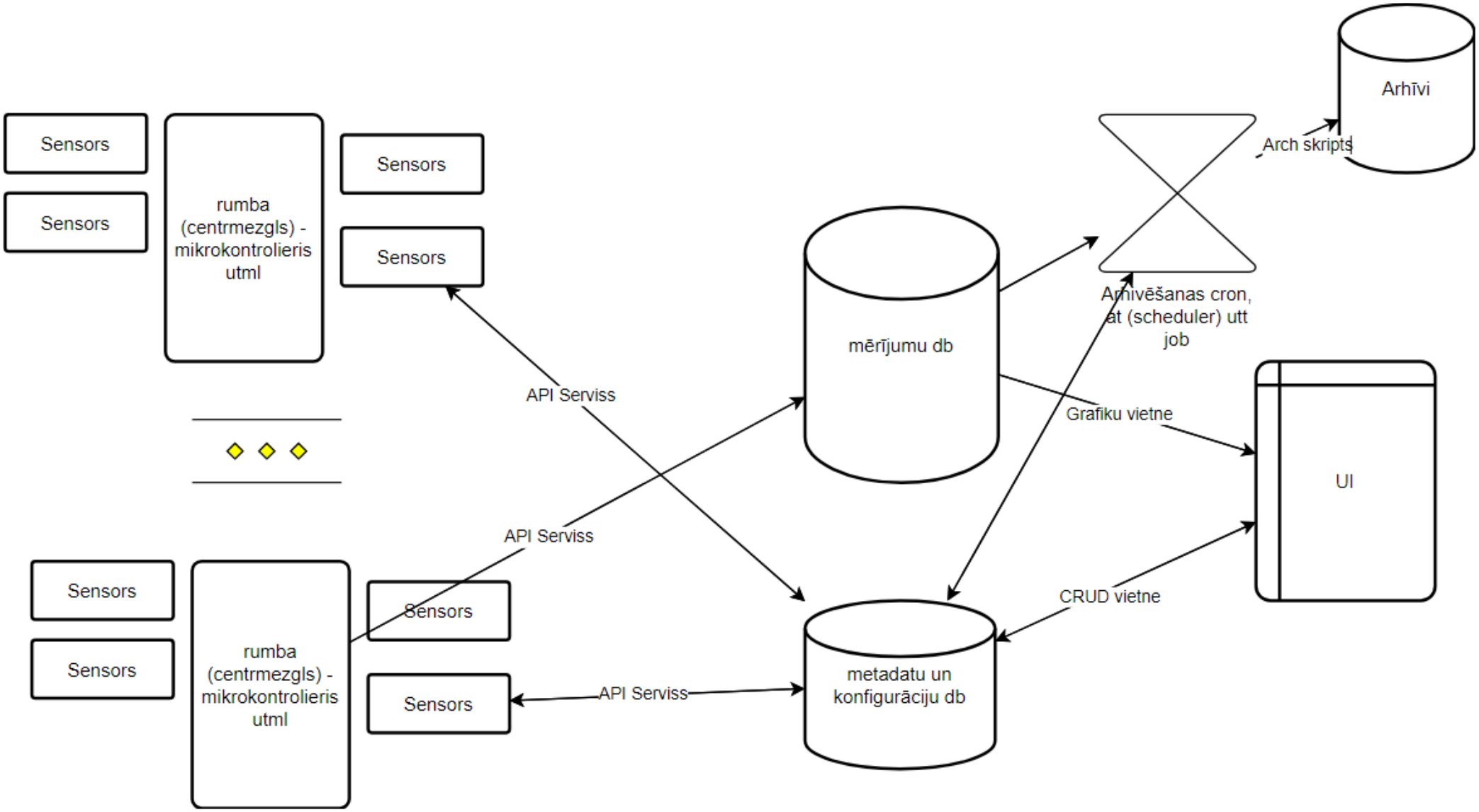
Nr.	Lietotāju stāsts <lietotājs> vēlas <sasniegt mērķi>, jo	Prioritāte	Izpilde
1.	Pasūtītājs vēlas savienotus sensorus ar rādītāju paneli, jo šī brīža risinājums nav pietiekami ērts.	A-> Must haves	J
2.	Pasūtītājs vēlas iespēju regulēt datu apmaiņas biežumu, jo ir jāatrod balanss starp biežumu, pārskatāmību un enerģijas patēriņu.	A-> Must haves	J
3.	Pasūtītājs vēlas rādītāju panelī redzēt sensora rādījuma interpretāciju ar prasībām publiskajā telpā, jo sensoru cipari nav intuitīvi / ātri uztverami	A-> Must haves	J
4.	Pasūtītājs vēlas rādītāju panelī iespēju regulēt glabāšanas laikus, jo laika gaitā mēdz mainīties pat likumdošana, kur nu vēl viedokļi par glabāšanas ilgumiem.	B-> Should haves	J
5.	Pasūtītājs vēlas trīs glabāšanas stratēģijas (db/arhīvs/dzēš), jo ir jāņem vērā veiktspēja arī pie bieža datu apmaiņas intervāla	B-> Should haves	J
6.	Pasūtītājs vēlas redzēt datus grafiski, jo cilvēkam dabiskāk un ātrāk uztverams ir zīmējums nekā apjomīgi skaitliski dati.	B-> Should haves	J
7.	Pasūtītājs vēlas iespēju meklēt/filtrēt sensorus atbilstoši atrašanās vietai, jo daudzu sensoru vidē nereti interesē konkrēts vai konkrēta to apakškopa.	B-> Should haves	J
8.	Pasūtītājs vēlas filtru sasaisti ar reālo telpas vietu, jo vizuāli ir ātrāk orientēties nekā ar simboliem.	C-> Could haves	N

3.2 Algoritms ↑

Sistēmu var ieadalīt 4 daļās:

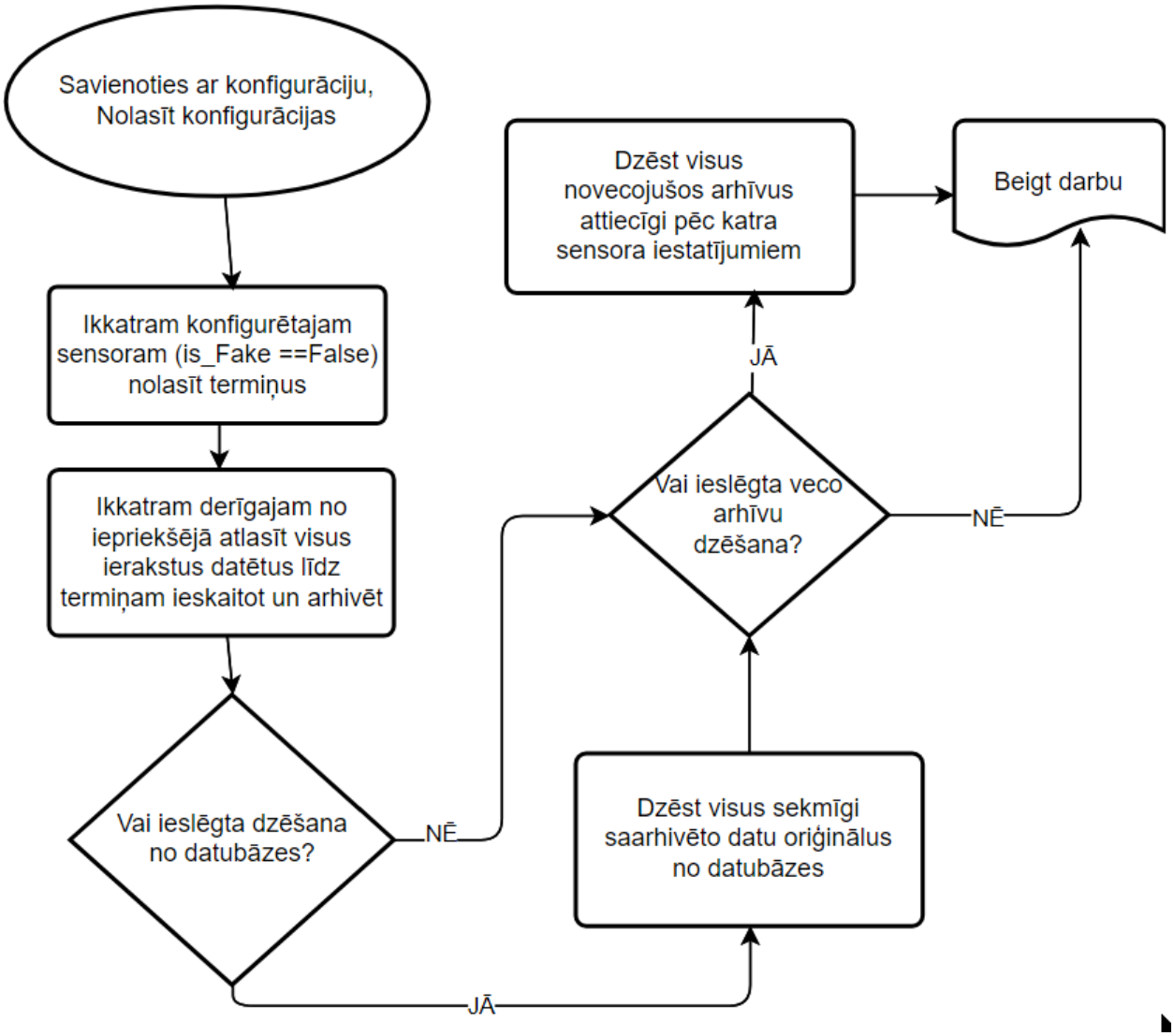
- mērījumu ievākšana (centrmezgli, mikrokontrolieri),
- mērījumu arhivēšanas un dzēšanas chron uzdevums,
- mērījumu ievākšanas saglabātāji (.py skripti uz 2 portiem),
- web GUI lietotājam (Go valodā rakstītā Grafana un CRUD operāciju Django).

Sistēmas kopskats: ↑

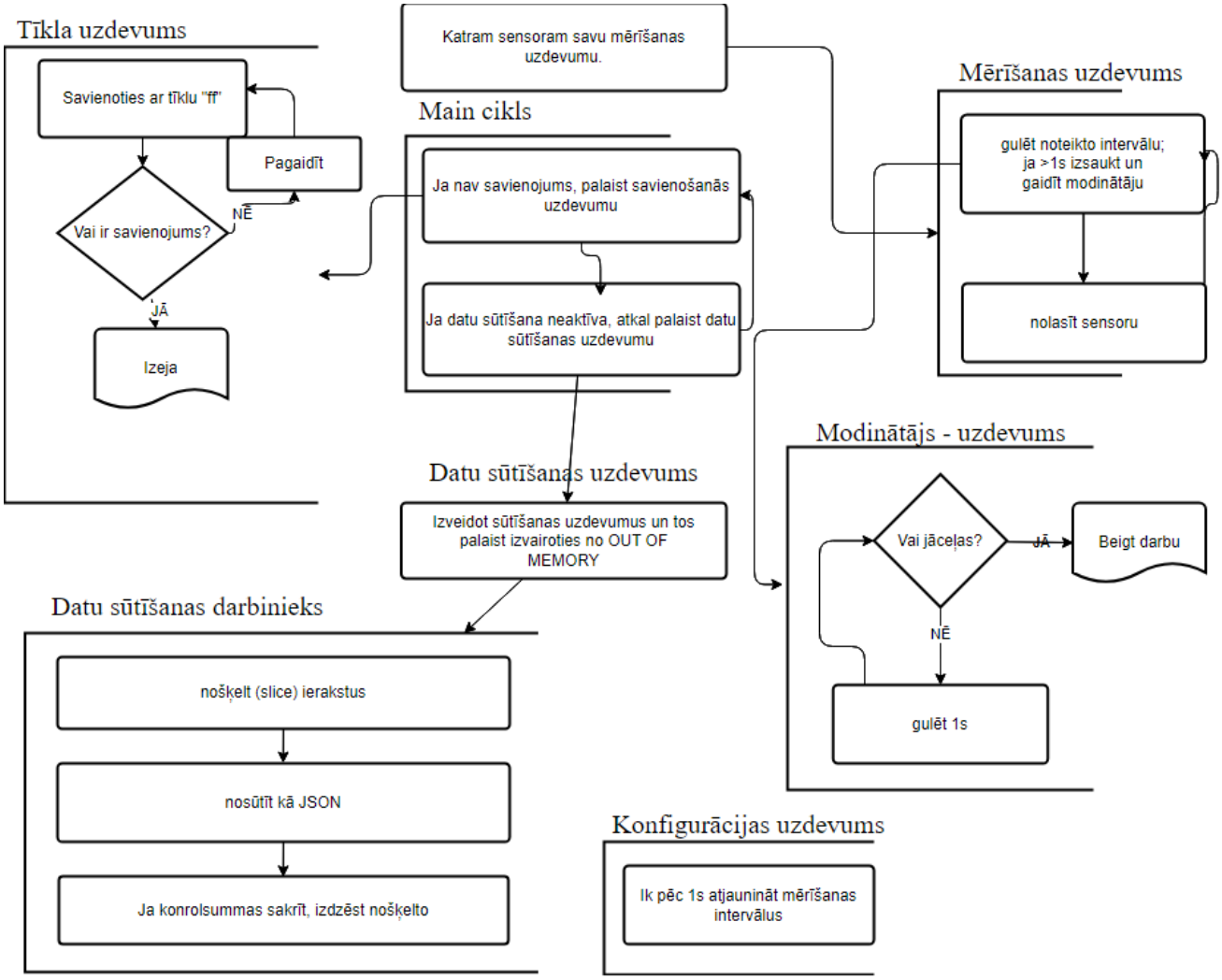


Arhivatora algoritms:

Dzēšana notiek ar panda dataframes palīdzību, jo arhivējot no sql tiek pārveidots csv un ar gzip saspiests.



Centrmezgla (mikrokontroliera) algoritms (katra grupa ir atsevišķs async tasks): ↑



Kalibrēšana 2B sensoram:

Bez nobīdes būtu: $V_{kal} = \frac{I_{mēr} * V_{ref}}{I_{rangs}}$

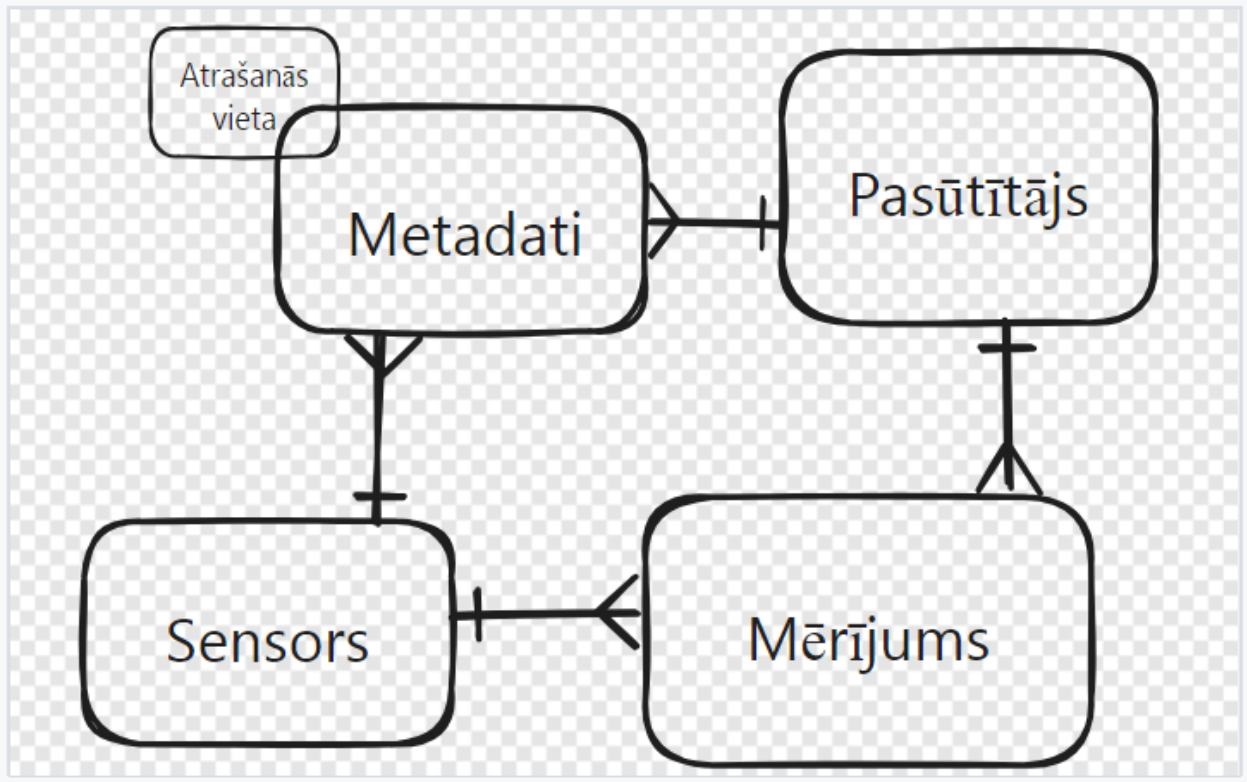
Ieejas ķēdes pretestību, kapacitāti,
Tāpēc "kalibrēju" ar ķēdē ieslēgtu s

Atsk nulli ignorēšu, aprēķinu pret V_{ref} teorētisko vism
linearitāte pat ar pārveidotāju kā starpnieku. Ja ne, at

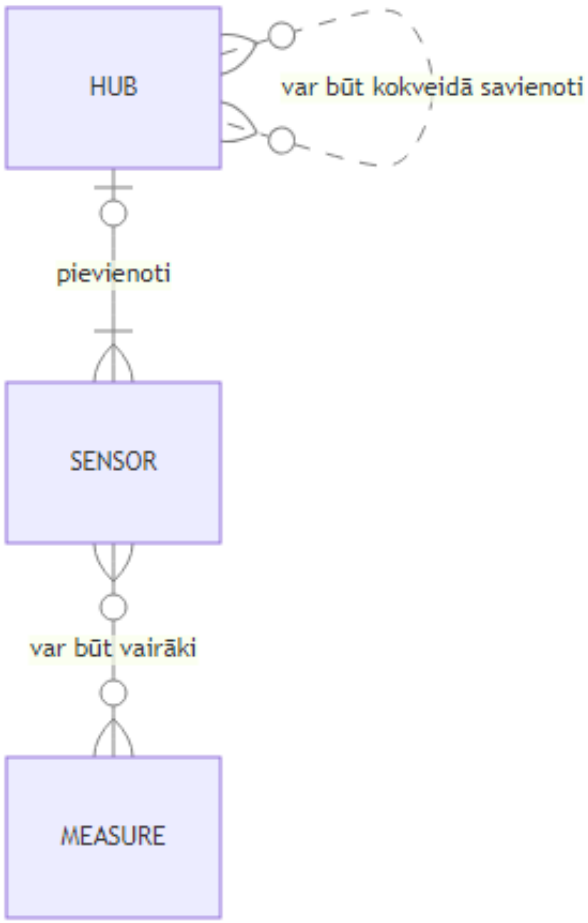
$$\Longrightarrow V_{m-kal} = \frac{I_{mēr} * 3300}{20} \Longrightarrow \qquad V_{m-kal} = I_{mēr} * 165$$

3.3 Konceptu modelis

Sistēmas:



Metadatu:



3.4 Tehnoloģiju steks

Satvari: Django/Micropython (mikrokontrolierim)/Grafana (grafiki)

Programmēšanas valoda: Python

Datu bāzes: MySQL/SQLite

Tīmekļa serveri: Apache (produkcijas)/Python Django (izstrādes)

Operētājsistēma: Ubuntu server

Virtualizācija: VirtualBox

4 Novērtējums

Lai novērtētu mēs pielietojām lekcijās doto metodi, proti, piešķiram kritērijiem koeficientus, tad katru kritēriju novērtējam kādā skalā (piemēram 10 baļļu) un tad aprēķinam svērto rezultātu. (svērtais novērtējums / maksimālais novērtējums)

Novērtējot ieguvām samērā augstu novērtējuma procentu. Tas liecina ka savus mērķus sasniedzām, pat neskatoties uz to, ka, piemēram, drošība ir novērtēta augstu, jo pievienot middleware, kas tāpat kā šībrīža middleware pārraksta noklusējuma prasību uz jebkuru simbolu un garuma atļaušanu lietotājvārdā un parolē, veiktu drošības prasību nodrošināšanu, ir samērā triviāls, mazāk nekā 2 cilvēkdienās paveicams uzdevums (protams, ja drošības algoritms jau ir python valodā realizēts gatavs).

Mūsu sistēmas MVP iekļauj sevī vairākus ISO25010:2011 aspektus:

Funkcionālie:

Sistēma ir labi piemērota sensoru mērījumu apskatei un pārraudzībai, kamēr vien tiek ievadīti realitātei atbilstoši metadati. Sistēmā ir iekļauti vairāki būtībā atšķirīgi sensoru tipi, tātad tā ir tuvināta reālajai situācijāi (lauka apstākļiem).

Sistēmas precizitāte ir atbilstoša konkrēto mikrokontrolieru precizitātei. Digitāliem sensoriem, protams tas ir iekšējs rādītājs, taču analogajam sensora piemēram mēs izmantojam 12 bitu ADC, kas ļauj iegūt 4096 vērtības.

Sistēma spēj sadarboties ar Raspberry pi, ja to lieto kā centrmezglu vai pat kā wifi nodrošinātāju pārējie centrmezgliem.

Drošība:

sistēmā ir iekļauts jaukšanas moments, kas neļauj ļaunprāšiem viegli paredzamā veidā iejaukties atjaunināšanas datos. Līdzīgus momentus iespējams jo viegli iekļaut arī sensoru datu pārraidē. Gadījumā ja pazūd tīkls, centrmezgli cenšas atkārtoti savienoties ar tīklu, un attiecīgi mērījumu ievākšana tiek apturēta, kad mērījumu saraksti aizņem tuvu kritiskajām atmiņas lielumus. Kad parādās tīkls, datu buferi tiek iztukšoti un mērījumi atsākti.

Funkcijas:

sasniedzām visas obligātās funkcijas, bet atmetām 3d vizualizāciju.

Sistēma atbilst tiem standartiem, kuru atbilstību ir deklarējuši attiecīgo iekārtu ražotāji. Analogajam mēram ir iekļauta rezerve gadījumiem kad strāva būtu virs 20mA, pie tam centrmezglam ir sava iekšējā aizsardzība pret īssavienojumiem.

Uzticamības: ↑

Sistēma pārbaudīti nodrošina savas funkcijas, tomēr ir samērā plašas izaugsmes iespējas.

Kā jau pieminēts, ir aizsardzība pret īssavienojumiem un rezerve augstākām strāvām. Tāpat jau minēts par tīkla darbības atjaunošanu.

Lietojamības:

Metadatu ievade ir attēlota ar pārbaudīto datatables saskarni, tāpat lietotāja grafiki ar pārbaudīto Grafana saskarni.

Mūsuprāt grafiki ir daudz pievilcīgāki par sākotnējo matplotlib versiju.

Efektivitātes:

Sistēma pēc ielādes ļauj responsīvi kārtot, šķirot metadatus. Ielādes laiku gan neizdevās uzlabot, jo pēc python3 atjaunināšanas kešošanai piedāvā tikai memcached vai redis, proti ārējus risinājumus.

Mikrokontrolieru atmiņa tiek lietota maksimālā līmenī, kā redzams OUT OF MEMORY video.

Uzturamības:

Kods ir samērā viegli un mazāk kā stundas laikā saslēdzams ar jebkuru centrmezglu, kas spēj darbināt micropython, jo mainās tikai porti un varbūt LED specifika.

Kustināmības (Portability):

Tāpat ir viegli paplašināmi metadati un pievienojami jauni objekti pēc nepieciešamības.

5 Secinājumi

Mums izdevās šajā īsajā laikā radīt samērā dzīvotspējīgu MVP Universitātes telpu gaisa kvalitātes pārraudzības sistēmu, kuru pateicoties tehnoloģiju izvēlei ir ļoti viegli mērogot vairāku kvalitātes kritēriju jomā, piemēram, drošības, uzticamības, kvalitātes.